

Real Estate Data Engineering Project Documentation

1. Project Overview

This project demonstrates an end-to-end modern data engineering solution using Azure services. The project extracts raw real estate data files from Azure Data Lake Storage Gen2, loads them into staging tables using Azure Data Factory (ADF), transforms the data into a dimensional model (Star Schema), and loads it into a Data Warehouse for reporting on property transactions, agent performance, leads, listings, offers, and viewings.

The project follows industry-standard ETL architecture and Medallion-style layered design.

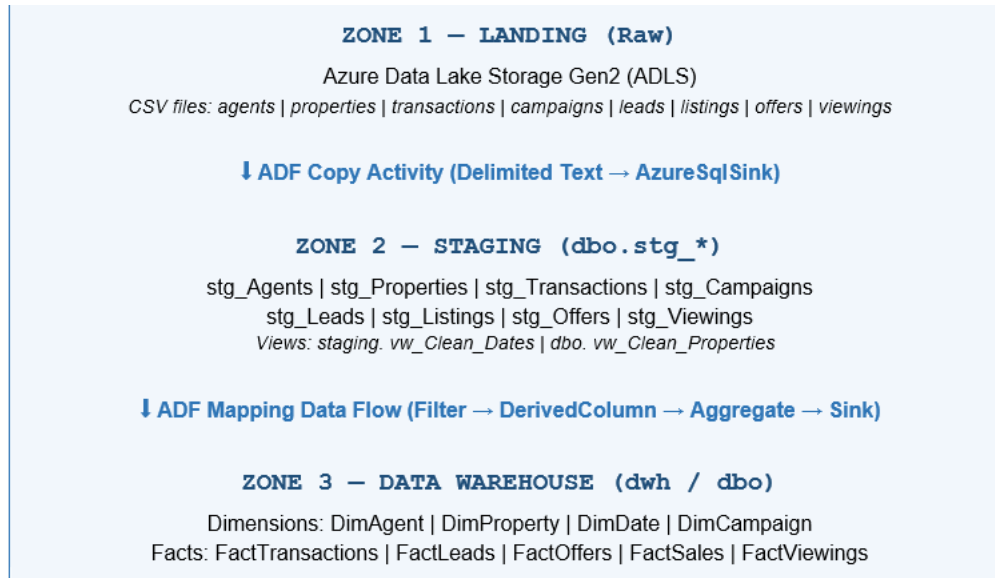
- Ingest raw CSV files (agents, properties, transactions, campaigns, leads, listings, offers, viewings) from ADLS.
- Stage data in Azure SQL Database (dbo schema) via ADF Copy pipelines.
- Clean, duplicate, and transform staged data using ADF Mapping Data Flows.
- Populate a star-schema data warehouse (dwh / dbo schemas) with dimension and fact tables.
- Enforce of — dimension and fact loads check row counts before insert and delete stale data before reload.

2. Technologies Used

Technologies		
Technology	Purpose	
Azure Data Factory	ETL orchestration	
Azure SQL Database	Staging + Data Warehouse	
Azure Data Lake Storage Gen2	Raw file storage	
SSMS	SQL development	
SQL Stored Procedures	Validation + preprocessing	
Mapping Data Flows	Data transformation	

3. Project Architecture

The system follows a classic three-zone medallion-inspired architecture layered inside Azure:

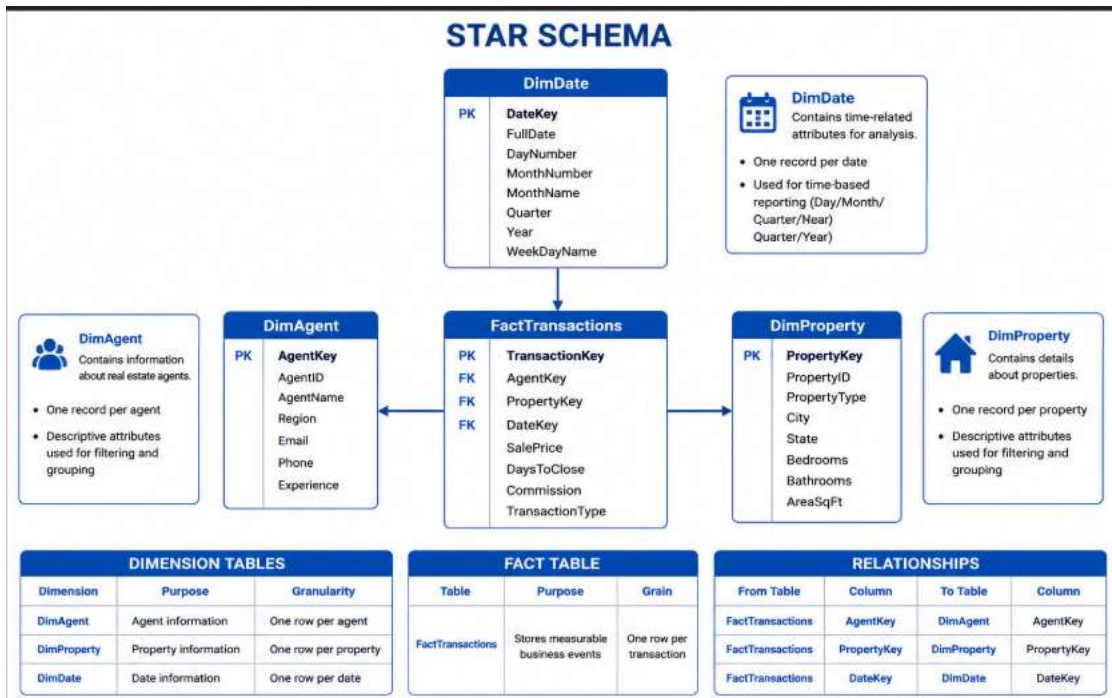


2.1 Linked Services

Two linked services connect ADF to the data stores:

Linked Service	Type
LS_ADLS	AzureBlobFS (ADLS Gen2)
LS_SQL	AzureSqlDatabase

STAR SCHEMA



3.1 Dimension Tables

DimAgent

Column Name	Data Type	Key
AgentKey	INT (IDENTITY)	PK
AgentID	VARCHAR	NK
AgentName	VARCHAR	
Region	VARCHAR	
City	VARCHAR	
JoinDate	DATE	
ExperienceYears	INT	

DimProperty

Column Name	Data Type	Key
PropertyKey	INT (IDENTITY)	PK

Int_1093

PropertyID	VARCHAR	NK
PropertyType	VARCHAR	
City	VARCHAR	
Region	VARCHAR	
Bedrooms	INT	
Bathrooms	INT	
AreaSqft	DECIMAL(10,2)	
YearBuilt	INT	

DimDate

Column Name	Data Type	Key
DateKey	INT	PK
FullDate	DATE	
Year	INT	
Quarter	INT	
Month	INT	
MonthName	VARCHAR	
Day	INT	

DimCampaign (defined, not yet ETL-loaded)

Column Name	Data Type	Key
CampaignKey	INT (IDENTITY)	PK
CampaignID	VARCHAR	NK
(other columns)	—	

3.2 Fact Tables

FactTransactions (primary — dwh schema)

Column Name	Data Type	Key
TransactionKey	INT (IDENTITY)	PK
AgentKey	INT	FK
PropertyKey	INT	FK
DateKey	INT	FK
SalePrice	DECIMAL(18,2)	
DaysToClose	INT	

Other Fact Tables (dbo schema — datasets defined)

Table	Purpose / Key Measures
FactLeads	Tracks marketing leads — lead source, campaign attribution, conversion status
FactOffers	Records buyer offers per listing — offer price, offer date, status (accepted/rejected)
FactSales	Completed property sales summary — sale price, agent, property, date
FactViewings	Property viewing events — viewing date, agent, property, outcome

4. Data Flow

Data moves through three logical phases, each orchestrated by ADF pipelines. The master pipeline (pl_master) sequentially calls each child pipeline using ExecutePipeline activities with waitOnCompletion=true, ensuring correct dependency ordering.

4.1 Phase 1 — Ingestion (ADLS → Staging)

Eight ADF Copy pipelines move CSV data from ADLS Gen2 into Azure SQL staging tables. Each pipeline contains a single Copy activity with a DelimitedText source and an AzureSqlSink.

Pipeline	Source (ADLS CSV)	Destination (Azure SQL)
pl_copy_agent	ds_adls_agent	dbo.stg_Agents
pl_copy_properties	ds_adls_properties	dbo.stg_Properties
pl_copy_transactions	ds_adls_transactions	dbo.stg_Transactions
pl_copy_campaigns	ds_adls_campaigns	dbo.stg_Campaigns
pl_copy_leads	ds_adls_leads	dbo.stg_Leads
pl_copy_listing	ds_adls_listings	dbo.stg_Listings
pl_copy_offers	ds_adls_offers	dbo.stg_Offers
pl_copy_viewing	ds_adls_viewing	dbo.stg_Viewings

4.2

Phase 2 — Dimension Loading (Staging → Dimensions)

Three pipelines load the three core dimensions. Each pipeline uses an idempotency check (Lookup activity) followed by conditional logic before executing a Mapping Data Flow.

PL_Load_DimAgent

- Executes data flow df_load_dim_agent directly (no pre-check for agents).
- Data flow steps: source stg_Agents → filter(AgentID IS NOT NULL) → derive(UPPER/TRIM name fields) → aggregate(GROUP BY AgentID, take FIRST of deduped values) → sink DimAgent.

- Rejected rows are stored back to ADLS via LS_ADLS linked service.

PL_Load_Dimproperties

- LKP_CheckPropertyCount: queries DimProperty row count from SQL.
- IF_PropertiesExist: if count > 0, skips load (no delete); otherwise executes data flow.
- Data flow df_load_dim_properties: source vw_Clean_Properties → derive (cast types) → aggregate(GROUP BY PropertyID, MAX Bedrooms/Bathrooms/AreaSqft) → sink DimProperty.
- SP_Validate_DimProperty: stored procedure validates the loaded dimension after insert.

PL_Load_DimDate

- LKP_CheckDates: checks row count in DimDate.
- If rows exist: sp_delete_dim stored procedure purges DimDate before reload (full refresh pattern).
- Data flow df_Load_dim_Date: source staging.vw_Clean_Dates (ClosingDate only) → derive DateKey=toInteger(yyyyMMdd), DayNumber, MonthNumber → aggregate(GROUP BY DateKey) → sink DimDate.

4.3 Phase 3 — Fact Loading (Staging + Dims → FactTransactions)

pl_load_fact_transactions

- LKP_CheckTransactions: queries row count in FactTransactions.
- If rows exist: SP_Delete_FactTransactions stored procedure truncates/deletes existing facts.
- Data flow df_Load_FactTransactions:

1. Source stg_Transactions — fields: TransactionID, ListingID, AgentID, PropertyID, OfferDate, ClosingDate, SalePrice
 2. Source DimAgent — AgentKey, AgentID, AgentName, Region, City, JoinDate, ExperienceYears
 3. Source DimProperty — PropertyKey, PropertyID, and all property attributes
 4. derivedColumn1: DateKey = toInteger(toString(ClosingDate,'yyyyMMdd'))
 5. join Agent (DimAgent JOIN stg_Transactions on AgentID — cross/auto broadcast)
 6. join1 (DimProperty INNER JOIN stg_Transactions on PropertyID)
 7. Sink dwh.FactTransactions (insert-only, skipDuplicates=true)
- Stored procedure1: post-load validation / metric logging stored procedure.

4.4 Master Pipeline Execution Order

Step	Pipeline	Depends On	Action
1	PL_Load_DimAgent	(start)	Load DimAgent dimension
2	PL_Load_Dimproperties	Step 1 Succeeded	Load DimProperty dimension
3	PL_Load_DimDate	Step 2 Succeeded	Load DimDate dimension
4	pl_load_fact_transctions	Step 3 Succeeded	Load FactTransactions fact table

5. BUSINESS RULES

1. SOURCE DATA

- All 8 CSV files must exist in Blob Storage before pipeline starts — Agents, Properties, Listings, Leads, Viewings, Offers, Transactions, Campaigns
- Files must have header row, comma delimiter, UTF-8 encoding
- Missing file = pipeline fails immediately

2. STAGING

- All 8 staging tables are TRUNCATED and fully reloaded every day
- All columns stored as VARCHAR in staging — no type casting here
- Every row gets _LoadedAt and _SourceFile audit columns stamped automatically
- No business logic or filtering in staging — raw data only

3. DATA QUALITY

- AgentID, PropertyID, ListingID, LeadID, ViewingID, OfferID, TransactionID, CampaignID — all mandatory, never NULL or blank
- No duplicate business keys allowed in any table
- All dates must be in yyyy-MM-dd format — invalid dates are rejected
- AskingPrice and SalePrice must be greater than zero
- OfferDate must always be earlier than or equal to ClosingDate
- LeadScore must be between 0 and 100
- Minimum row count must pass after staging — if Agents < 400, Transactions < 7000 etc., pipeline stops

4. DIMENSIONS

- DimAgent, DimProperty, DimListing, DimCampaignChannel — 4 dimension tables
- All use surrogate integer keys (IDENTITY) as primary key
- SCD Type 1 — old values are overwritten, no history kept
- Load order is strict — DimAgent + DimProperty first (parallel), then DimListing, then Facts
- Agent Tenure Band — Junior (0–1yr), Mid (2–4yr), Senior (5–9yr), Expert (10+yr)
- Property Size Band — Micro (<500sqft), Small (500–999), Medium (1000–1999), Large (2000–3999), XLarge (4000+)
- Days on Market — ListDate to EndDate; if listing still active, uses today's date
- Unknown campaign channel defaults to 'Unknown' — never rejected

5. FACTS

- 5 fact tables — FactTransaction, FactLead, FactViewing, FactOffer, FactCampaign
- Facts are incremental — only new records inserted, no truncate-reload on facts
- All date keys stored as YYYYMMDD integer to join with DimDate
- Price Delta = SalePrice minus AskingPrice (negative = sold below asking)
- Discount % = $(\text{AskingPrice} - \text{SalePrice}) / \text{AskingPrice} \times 100$
- Days to Close = OfferDate to ClosingDate in FactTransaction
- Lead Score Bands — Cold (0–9), Warm (10–24), Hot (25–100)
- Viewing Feedback — IsPositive and IsNegative are auto-computed binary flags
- Offer Status — IsAccepted, IsRejected, IsPending are auto-computed binary flags
- Campaign Duration = StartDate to EndDate in days

6. DATE DIMENSION

- DimDate covers 2015 to 2030 — pre-populated, no pipeline needed
- Any event date outside this range gets NULL DateKey
- IsWeekend flag = 1 for Saturday and Sunday

7. PIPELINES

- 5 pipelines total — PL_Ingest_Staging → PL_Validate_Staging → PL_Load_Dimensions → PL_Load_DataFlows → called by PL_Master_Orchestrator
- Strict sequence — next pipeline only starts if previous one succeeds
- Any failure stops everything, logs error, sends alert email
- Every pipeline writes Start and End entry to log.PipelineAudit
- Runs daily at 01:00 UTC automatically
- Pipeline is idempotent — safe to rerun same day without duplicating data

8. DATA FLOWS (ADF)

- 8 Mapping Data Flows — one per source entity
- Each flow does — Source → Type Cast → Filter nulls/bad data → Lookup surrogate keys → Sink to DWH
- Transactions, Leads, Viewings, Offers, Campaigns all do anti-join before insert to prevent duplicates
- Listings flow joins both DimAgent and DimProperty to resolve surrogate keys before loading